

PROFILE SUMMARY

- Senior Rust Developer with 8 years of experience building memory-safe systems Rust at infrastructure scale across virtualization platforms, container runtimes, and Linux distribution tooling, specializing in ownership and lifetime design, Tokio async runtimes, and unsafe FFI with safety invariants.
- Hands-on coverage across Rust (Rust 2021 edition), async runtime (Tokio), web stack (Axum), persistence (SQLx, PostgreSQL), and Wasm and embedded (wasm-bindgen, clap) with strong fundamentals in ownership and borrowing, traits and generics, Result and Option handling, and a working command of pinning, async traits, and lifetimes.
- Deep expertise in ownership and lifetime modeling, async Tokio task scheduling, unsafe FFI with safety invariants, and zero-cost abstractions and SIMD tuning, applying methodologies such as working with the borrow checker over fighting it and proptest, miri, and cargo-fuzz coverage on unsafe code to deliver memory-safe, low-latency Rust services that hold the line under production load.
- Engaged collaborator working cross-functionally with Kernel, Security, Platform, and SRE teams in open-source-first and async-friendly remote teams, contributing to RFC threads, on-call rotations, and architecture reviews with an ownership-first mindset and clean handoffs.
- Mentor who shares technical excellence and fosters a culture of memory safety and performance discipline through PR reviews and pattern docs, while running Rust guild and unsafe-review sessions and authoring widely used crate and unsafe-block templates.

TECHNICAL SKILLS

Languages & Core Rust:	Rust 2021 edition, Rust 2024 edition, traits, generics, lifetimes, pattern matching, enums, Result, Option, error handling with thiserror and anyhow, macros
Async & Concurrency:	Tokio, async-std, smol, futures, pinning, async traits, channels, Arc, Mutex, RwLock, atomics, Rayon, crossbeam
Web & Backend:	Axum, Actix Web, Rocket, Warp, Tower, hyper, reqwest, tonic, SQLx, sea-orm, Diesel, PostgreSQL, Redis, OpenAPI
Systems & Performance:	unsafe Rust, raw pointers, MaybeUninit, perf, flamegraph, cargo-flamegraph, criterion, hyperfine, SIMD, std::arch, zero-cost abstractions, cache-friendly layouts
Wasm, CLI & Embedded:	wasm-bindgen, web-sys, Wasmtime, Wasmer, Extism, Cloudflare Workers, Spin, clap, ratatui, no_std, embassy, RTIC, cortex-m
Testing, Tooling & Quality:	cargo test, criterion, proptest, quickcheck, cargo-fuzz, libFuzzer, AFL, clippy, rustfmt, miri, cargo audit, cargo deny, cargo workspaces
Interop, FFI & Polyglot:	bindgen, cbindgen, PyO3, maturin, NAPI-RS, neon, cxx, JNI, FFI safety patterns, Send and Sync, Pin and Unpin, repr(C)
Build, Toolchain & CI/CD:	Cargo, rustup, cross-compilation, musl static binaries, Docker distroless, GitHub Actions, GitLab CI, semver release flow, tracing, OpenTelemetry

EDUCATION

Technische Universität Berlin M.S. in Computer Science Berlin, Germany • Sep 2013 - Jun 2018

WORK EXPERIENCE

AWS (Bottlerocket) Senior Rust Developer Berlin, Germany (Remote) • May 2021 - Present

- Owned idiomatic Rust development end to end on the container-optimized Linux distribution for EKS and ECS running

across **2.4M hosts**, shipping **update agent**, **API server**, and **bootstrap controller** across **24 crates** held to the Rust API guidelines and the project's own RFC review bar.

- Mastered ownership, borrowing, and lifetimes with **borrowed views over owned config trees with explicit 'a lifetimes**, Cow for borrowed-or-owned data flow, and Box and Rc only where the model demands it, removing **312 unnecessary clones** and dropping workspace compile time from **9.4min** to **2.8min**.
- Owned memory safety and unsafe Rust through **audited unsafe blocks around raw syscalls and FFI with documented safety invariants**, a minimal blast-radius policy that keeps every unsafe scope under ten lines, and module-level SAFETY comments on each one, auditing **84 unsafe blocks** and holding miri coverage on the unsafe surface at **100%**.
- Drove async programming and concurrency on **Tokio multi-threaded runtime with bounded mpsc channels and Arc<RwLock> shared state**, tokio::select for cancellation, and Rayon for the data-parallel diff stage, capping concurrent tasks at **4096** and dropping lock contention on the hot path from **38%** to **4%**.
- Tuned systems and performance with **cargo-flamegraph profiling with criterion regression gates and cache-friendly layouts**, perf event sampling, and SIMD via std::arch on the verification path, dropping p99 update apply latency from **820ms** to **95ms** and cutting heap allocations on the hot path by **74%**.
- Built the web and back-end control plane on **Axum with Tower middleware, SQLx connection pooling, and tonic for gRPC control plane**, hyper for low-level transport, and reqwest for outbound calls, shipping **110+** endpoints behind a typed handler layer and holding service uptime at **99.99%** across the last four quarters.
- Shipped WebAssembly modules with **Wasmtime-hosted policy plugins with Extism plugin SDK and capability-based sandboxing**, wasm-bindgen for the operator console UI, and a thin host ABI documented per release, landing **32 modules** in production and cutting plugin cold-start from **180ms** to **14ms**.

Mozilla [Software Engineer \(Rust\)](#)

Berlin, Germany • Jul 2018 - Apr 2021

- Drove testing, tooling, and code quality with **proptest property tests, cargo-fuzz harnesses, and criterion regression gates**, clippy and rustfmt on every PR, and miri runs on the unsafe surface, dropping crash-bug rate per release from **42** down to **6** and lifting cargo-fuzz coverage on the parser surface to **91%**.
- Built CLI and embedded-leaning tooling with **clap-driven CLIs cross-compiled to 6 targets with musl static binaries**, a ratatui-based debugger TUI, and no_std prototypes on cortex-m boards, shipping across **6 targets** and shrinking the release binary from **42MB** to **4.2MB**.
- Owned interop and FFI work through **PyO3 bindings exposed via maturin and cbindgen-generated C headers for the C++ host**, NAPI-RS shims for the Node tooling, and a documented Send and Sync policy on every shared type, bridging **4 host languages** without a single use-after-free regression over the run.
- Strengthened idiomatic Rust foundations across the team through **trait-based design, Result and Option flows, and newtype patterns over raw primitives**, module layout reviews, and Rust Book reading sessions, while mentoring **5 engineers** through pair-programming, code-review checklists, and a written Rust idiom guide adopted across two squads.